TDP-Reasoner: Timing-Dependent Plasticity Along the Recursion

Anonymous Author(s)
Affiliation
email

Abstract

Small recursive models trained from scratch are the strongest non-LLM approach to puzzle benchmarks such as Sudoku-Extreme and ARC-AGI. These models, HRM and TRM among them, re-apply a Transformer block many times. Seen as a modern Hopfield network, each step's attention is a Hebbian memory of the current state, so recursion reinforces the model's current answer. We propose TDP-Reasoner (timing-dependent plasticity reasoner), which keeps a fast-weight memory along the recursion itself: keys and values from different steps are paired under a learned two-sided timing window, borrowed from spike-timing-dependent plasticity, so the memory tracks how the state moves rather than where it is. Demonstrations of a task reach the query only through a per-task linear rule fitted in context, so unseen tasks need no retraining. Run 39 times deeper than it was trained, it removes 55% of its remaining errors on held-out Sudoku-Extreme puzzles and never loses a solved puzzle. On unseen ARC-AGI-1 tasks with small grids it matches the public TRM checkpoint (43.2% vs. 45.5%) after about 3% of its training; at full scale it reaches 18.8% pass@2 on ARC-AGI-1 (TRM: 41.8% under the same evaluation) and 0% on ARC-AGI-2.

1 Introduction

Small recursive models such as HRM [1] and TRM [2] apply one Transformer block many times. The recursion creates a time axis, but nothing crosses it except the state: every step rebuilds its attention from the current state alone. Seen as a modern Hopfield network [3], that attention is a Hebbian memory of the current state, so each step reinforces the answer the model already holds. On ARC, extra recursion changes little: the public TRM checkpoint gains nothing when run past its training depth [8], and TRM models trained on ARC settle on wrong fixed points [5].

Synapses in the brain learn from the order of events as well as their coincidence: under spike-timing-dependent plasticity (STDP), the sign and size of a change depend on which neuron fired first and by how much [6, 7]. We apply this rule along the recursion. A fast weight pairs keys and values from different steps under a learned two-sided timing window, so the memory records how the state moves, not only where it is. No spikes are needed, because the window uses only the order and lag of two events, which the step index provides.

For ARC, TRM receives the demonstrations of a task through a puzzle embedding learned for that task, so an unseen task has to be trained on before it can be solved [8, 9]. TDP-Reasoner instead fits a sparse linear rule to the demonstrations in context and refines it at every step of the recursion.

Our contributions are (i) timing-dependent plasticity along the recursion depth, whose memory equals the pair-based STDP integral exactly; (ii) an in-context rule head that keeps prior knowledge in slow features and the task in a fast sparse map; and (iii) results on Sudoku-Extreme extrapolation and on ARC-AGI-1 and ARC-AGI-2, where TDP-Reasoner scores 0%.

2 Related work

[To be written.]

3 TDP-Reasoner

Block. One block with shared weights is applied at recursion steps $n = 1, 2, \ldots$. Each head reads

$$\text{read}_n = \left(\textstyle\sum_\tau v_{n,\tau} \tilde{k}_{n,\tau}^\top + W_n \right) \tilde{q}_n \tag{1}$$

with unit-length, rotary-encoded queries and keys (QK-norm); the attention and a SwiGLU feed-forward layer each add a residual followed by a parameter-free RMSNorm. Without W_n this is recurrent linear attention over the tokens τ, which keeps nothing across steps but the state.

Memory across the recursion. W_n is a fast weight carried across steps. Each token keeps decaying traces of its keys and values, $\bar{k}_r \leftarrow z_r \bar{k}_r + \gamma_r \tilde{k}$ and $\bar{v}_r \leftarrow z_r \bar{v}_r + \gamma_r v$, over $R = 4$ bands with learned complex poles $z_r = \exp(-e^{\nu_r} + i\theta_r)$ and $\gamma_r = \sqrt{1 - |z_r|^2}$, which keeps them bounded. Each step pairs the new value with the key traces and the new key with the earlier value traces,

$$W_n = (1 - \eta)\, W_{n-1} + \eta \sum_\tau \sum_r \text{Re}\big(a_r\, v_n \bar{k}_{r,n}^\top + b_r\, z_r \bar{v}_{r,n-1}\, \tilde{k}_n^\top \big), \tag{2}$$

with learned complex a_r, b_r and rate $\eta \le 1/8$. Unrolled, exactly,

$$W_n = \sum_\tau \sum_{s,t \le n} \eta (1 - \eta)^{n - \max(s,t)}\, L(t - s)\; v_{t,\tau}\, \tilde{k}_{s,\tau}^\top, \qquad L(\Delta) = \begin{cases} \sum_r \gamma_r \,\text{Re}(a_r z_r^\Delta) & \Delta \ge 0, \\ \sum_r \gamma_r \,\text{Re}(b_r z_r^{|\Delta|}) & \Delta < 0. \end{cases} \tag{3}$$

With the key as presynaptic and the value as postsynaptic activity, this is the pair-based STDP integral: every pair of events is added once, when the later one arrives, weighted by a window of their lag whose two sides separate key-first from value-first pairs. The exponential STDP window is the case $R = 1$, $\theta = 0$, $a > 0$, $b < 0$. No spikes are needed: in a recursion like this, spiking itself carries no information of substance, and the window uses only the order and lag of two events, which the step index supplies. At a fixed point all pairs are equal, and W reduces to a Hebbian term scaled by the area of the window. *[Figure: windows learned on Sudoku.]*

This is the Sudoku model: recurrent linear attention with QK-norm, plus traces and a timing window.

Reading demonstrations. For ARC, each example is a lane processed on its own, with its output grid hidden; the lanes of a task form a group. A rule head reads the demonstrations through one per-task linear map from learned cell features $\phi = W_\phi h / \|W_\phi h\|$ to colours y,

$$\Theta = \arg\min_\Theta \sum_{o \in \text{demos}} \big\| y(o) - \Theta\, \phi(o) \big\|^2 + \lambda \|\Theta\|_1, \qquad r(o) = \Theta\, \phi(o), \tag{4}$$

and feeds r into the next block on the output canvas. Prior knowledge about ARC lives only in the slow features ϕ. The rule is chosen by a fixed principle that does not depend on the task: the sparsest linear map that fits the demonstrations. The task itself is held only in the fast Θ. No task is stored in the weights, so an unseen task needs no puzzle embedding and no test-time training. Queries never enter (4), and each demonstration is fitted on the others. Each block takes one proximal-gradient step on (4), so the rule is refined along the recursion together with the state.

Training and inference. A segment runs a fixed number of blocks and backpropagates through the last few. State, traces, W and Θ carry over to the next segment without gradient, with a stablemax cross-entropy loss after every segment (TRM's deep supervision, without halting). At inference each task runs in several dihedral and colour-permuted versions whose answers are voted; ties are broken by how well each version predicts its own demonstrations.

4 Experiments

[To be written.]

5 Limitations and future work

[To be written.]

References

[1] Wang et al. Hierarchical Reasoning Model. arXiv:2506.21734, 2025.

[2] Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. arXiv:2510.04871, 2025.

[3] Ramsauer et al. Hopfield Networks is All You Need. ICLR, 2021.

[4] Li et al. Scaling to Tens of Thousands of Test-Time Iterations with Loop-Native Attention Residuals. arXiv:2610.11570, 2026.

[5] Viakhirev et al. Think Shallow, Solve Deep: Controlling Recurrent Dynamics for Reliable Test-Time Depth. arXiv:2608.18222, 2026.

[6] Markram, Lübke, Frotscher, Sakmann. Regulation of Synaptic Efficacy by Coincidence of Postsynaptic APs and EPSPs. Science 275:213–215, 1997.

[7] Bi, Poo. Synaptic Modifications in Cultured Hippocampal Neurons: Dependence on Spike Timing, Synaptic Strength, and Postsynaptic Cell Type. J. Neurosci. 18:10464–10472, 1998.

[8] Roye-Azar et al. Tiny Recursive Models on ARC-AGI-1: Inductive Biases, Identity Conditioning, and Test-Time Compute. arXiv:2512.11847, 2025.

[9] McGovern. Test-time Adaptation of Tiny Recursive Models. arXiv:2511.02886, 2025.